Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

Simple Hospital appointment system.

2. Problem you want to solve (2–3 short sentences):

Manual booking is error-prone & time-conpuming. Hospital needs easy way to manage.

3. Target users (who will actually use this?):

Hospital receptionists, patients.

4. Inputs (what data / actions will the program receive?):
   - patient details.
   - Appointment date & time, doctor schedule
5. Outputs (what will the user see / get?):
   - Confirmation of appointment booking.
   - Appointment reminders
6. Constraints or assumptions (keep it realistic):
   - SMS / email service is simulated.
   - Users should have basic computer literacy.

Part B – Core features vs "nice-to-have"

1. List minimum features needed for a working version (MVP):
   - Add a new appointment.
   - View appointments by doctor.
   - Send appointment confirmation to patient.
2. List optional features (only if time allows):
   - Automated reminder SMS / email.
   - User login & role-based access
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

I would keep "Add, view, cancel / reschedule appointments" basically it delivers core functionality of managing appointments.

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal

Learn to split a project into logical modules and functions.

Instructions

Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: data input
   - Module 2: Appointment processing
   - Module 3: Reporting & Notifications
   - Module 4: User interface
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Handles collection & validation of patient.
   - Module 2 does: Manages booking, cancellation of appointments
   - Module 3 does: Generates reports & send confirmation
   - Module 4 does: provides an interactive interface for recep-tionists.

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: Hospital appointment processing.
2. Functions planned:
   - Function 1 name: book appointment.
     - Input(s): patient information, doctor id, date & time
     - Output: appointment id.
   - Function 2 name: Cancel appointment
     - Input(s): appointment id.
     - Output: Status
   - Function 3 name: reschedule appointment.
     - Input(s): appointment id, new date & time
     - Output: updated appointment id
3. Draw or briefly describe how data flows between these functions:

Book appointment - receives patient & doctor data, create an appointment record, & passes the appointment id.

Cancel appointment - takes the appointment id, removes the record & sends a status message.

reschedule appointment - updates the existing record with new time & returns the updated ID.

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

   patient record.

2. For this entity, list required fields:

   - Field 1: patientID (type: int/str/float/...)
   - Field 2: name (type: str)
   - Field 3: age (type: int)
   - Field 4: diagnosis (type: str.)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

```
patient item = {
    "field1": 123,
    "field2": _____,
    ...
}
```

```
patient = {
    "patient_id" : 123,
    "name" : "Ashwini NM",
    "age" : 21,
    "diagnosis" : "flu"
}.
```

2. Option B – Class:

python

```
class Item:
    def __init__(self, field1, field2, ...):
        self.field1 = field1
        self.field2 = field2
        ...
```

```
2.
def __init__(self, patient id, name, age, diagnosis):
    Self.patient_id = patient_id.
    Self.name = name.
    Self.age = age
    Self.diagnosis = diagnosis.
```

3. Which option will you use, and why?

   Option B, use the class)

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
```

```
patient = patient record (123, "Ashwini NM", 21, "flu")
print (patient.name).
print (patient.age).
def display_summary (self):
```

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal

Build habit of designing tests before writing code.

Instructions

Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: Calculate discount price
2. In simple words, what should this function do?

3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s)	Expected output
1	price = 100 discount = 10	90.0
2	price = 200 discount = 0.	200.0
3	price = 150, discount = 25	112.5
4	price = 50. discount = 100	0.0
5	price = 0 discount = 20	0.0

4. Now write a small test harness:

python

```
def my_func(...):
    # TODO: implement
    pass

# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

```
def calculate_discounted_price (price, discount):
print ("Test 1:", calculate_discounted_price (100,10),
        "expected:", 90.0).
print ("Test 2:", calculate_discounted_price (200,0),
        "expected:", 200.0).
print ("Test 3:", calculate_discounted_price
        (150, 25), "expected:", 112.5).
print ("Test 4:", calculate_discounted_price
        (50, 100), "expected:", 0.0)
print ("Test 5:", calculate_discounted_price
        (0, 20), "expected:", 0.0).
```

5. Reflection:

- Did writing tests first change how you thought about the function? How?

writing the test cases first forced me to think about function's behaviour in detail

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: File not found.
   - How to detect it? Attempt to open file
   - How to handle it politely for the user? "File not found. please check file path & try again"
2. Failure Point 2: permission error.
   - Detect: File open fails with permission error.
   - Handle: Show "permission denied. Make sure you have read access to the file"
3. Failure Point 3: Invalid file format.
   - Detect: Try to read the file content & catch check-file extension
   - Handle: Inform the file format is invalid / corrupted.

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")

try:
    with open(file_path, "r") as f:
        data = f.read()
    print("File loaded successfully.")
except FileNotFoundError:
    print("File not found. Please check the path.")
except PermissionError:
    print("Permission denied. Try a different file.")
```

file_path = input("enter file path:")
try:
with open (file-path, 'r', encoding = 'utf-8') as f:
data = f.read()
print ("file loaded successfully.")
Except File Not found error.

Adapt this pattern to your project and paste your modified code below:

python

 # Your robust code:

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal

Practice cleaning up messy, working code into clear, modular code.

Instructions

Take one working script (from your own work or given by teacher).

It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

   Code name - data_processor.PY.

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: long monolithic function.
   - Problem 2: code repetition.
   - Problem 3: confusing variable names.

3. Plan refactoring steps (high level):
   - Step 1: Break the monolithic function into smaller, single responsibility
   - Step 2: Remove repetation by creating reusable helper
   - Step 3: Rename variables

4. After refactoring, write or paste your improved version here:

python

```
# Refactored code:
import csv.
def read_csv.:
    with open (file_path, 'r') as f:
    return list (csv.DictReader (f)).
def calculate_sales :
  total_sales = sum (float (row ['sale']) for row in data)
    avg_sale = total_sales / len (data)
    return total_sales , avg_sale.
def display_results
    print (f" Total sales : {total_sales :.2f}")
    print (f" Average sale : {avg_sale :.2f}")
def main ():
        file_path = 'sales.csv'
        sales_data = read_csv
        total, average = calculated_sales
        display_results (total, average)
```

5. Reflection:
   - What became easier to understand after refactoring?

   The code's logic is now split into clear, named functions, making it simple to see each step.

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?

   The tool should add a student record with a name & score to a data store.
3. Sketch CLI arguments:

Argument / Option	Meaning
action	Command
-name	Student's name
- Score	Student's Score

4. Implement with argparse (skeleton):

python

```
import argparse

parser = argparse.ArgumentParser(description="Describe your tool here")
parser.add_argument("--name", type=str, help="Student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()

# Use args.name, args.score here
print("Name:", args.name)
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

   Add ur project - specific logic where the print statements are & include example usage at the bottom of the file.

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal

Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Requirements gathering & feature specification
   - Milestone 2: design prototype
   - Milestone 3: core functionality development
   - Milestone 4: Module testing & bug fixing.
   - Milestone 5: User acceptence testing.
2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
Requirements	Signed-off feature specification	Week 1
design	Interactive prototype.	week 2.
Core development	Functional alpha version	Week 4.

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: Technical complexity.
     - Plan B if this fails: prepare full back local implementation.
   - Risk 2: Time slippage.
     - Plan B if this fails: prioritize core feature
   - Risk 3: User feedback issue.
     - Plan B if this fails: Conduct early usability tests

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

I broke the problem into smaller tasks & searched online.

2. One thing I now believe about myself as a problem-solver:

I realize I can handle complex technical challenges.

3. If I start a new project tomorrow, what will I do differently from day 1?

I will set clearer milestone checkpoints & allocate more time for risk-mitigation planning.